

# [项目实战]物流-数仓搭建阶段

## 练习题

### 习题一 解决 HiveServer2 内存溢出问题

请尝试用 GPT 探索 hiveserver2 内存溢出的解决方案。

### 习题二 编写数据装载脚本

请尝试用 GPT 编写原始数据到 hive ods 层的装载 bash 脚本。

### 习题三 辅助编写 SQL

请尝试使用 GPT 编写项目中的 SQL（比如卡车主题）

## 参考答案

### 习题一 解决 HiveServer2 内存溢出问题

#### 第 1 轮提示

#### 提示

我是用 datagrip 连接 hiveserver2 进行查询，hiveserver2 后台频繁出现 outofMemoryError，该怎么办

## 答

**OutOfMemoryError** 通常是由于 Java 虚拟机没有足够的空间来分配对象导致的，这在处理大数据时是一个常见的问题。解决这个问题通常需要增加 Java 虚拟机的可用内存。

以下是一些可能的解决方案：

1. **增加 JVM 内存：** 你可以尝试为 HiveServer2 提供更多的内存。这可以通过在 `hive-env.sh` 中设置 **HADOOP\_HEAPSIZE** 和 **HIVESERVER2\_OPTS** 来实现。例如：

```
export HADOOP_HEAPSIZE=2048
export HIVE_SERVER2_OPTS="-Xmx4g"
```

2. 这个设置将 Hadoop 的堆大小设置为 2GB，将 HiveServer2 的最大堆大小设置为 4GB。这些值需要根据你的硬件条件来调整。
3. **优化 Hive 查询：** 如果你的查询是非常大的，可能需要优化你的 Hive 查询来减少内存的使用。这可能涉及到使用更多的分区，避免笛卡尔积，使用适当的文件格式，等等。
4. **使用 Tez 执行引擎：** 如果你目前使用的是默认的 MapReduce 执行引擎，可以尝试切换到 Tez 执行引擎，它对内存的使用更加高效。
5. **升级硬件：** 如果你的硬件条件允许，可以考虑升级你的硬件来提供更多的内存。

记住，你做出任何更改后都需要重新启动 HiveServer2 服务。如果你的问题依然存在，你可能需要寻求专业的帮助，或者深入研究你的查询和数据来找到最佳的解决方案。

## 反馈

这个解决方案是正确的。

## 习题二 编写数据装载脚本

### 第 1 轮提示

#### 提示

#### 答

好的，了解您的需求了。下面是一个简单的 bash 脚本。需要注意的是，由于我不知道您的环境配置的具体情况，因此您可能需要根据实际情况对该脚本进行一些调整。

脚本涉及到日期计算、命令行参数处理、Hive 数据加载等多个部分。如果有任何问题，欢迎随时向我提问。

```
#!/bin/bash

# 获得昨天的日期
function get_yesterday_date() {
    date -d "yesterday" +%Y-%m-%d
}

# 检查 HDFS 路径下是否有文件
function check_hdfs_file() {
    hdfs dfs -test -e $1
    if [ $? -ne 0 ]; then
        echo "No such file or directory: $1"
        exit 1
    fi
}

# 将数据 load 到 Hive 中
function load_data_to_hive() {
    tablename=$(echo $1 | cut -c5-)
    hive -e "LOAD DATA INPATH '$2' INTO TABLE $tablename PARTITION (dt='$3')"
```

```
# 声明配置变量 table_list
table_list=("table1" "table2" "table3")

# 默认的 date 为昨天
date=$(get_yesterday_date)

# 解析命令行参数
while (( "$#" )); do
    case "$1" in
        --date)
            if [ -n "$2" ] && [ ${2:0:1} != "-" ]; then
                date=$2
                shift 2
            else
                echo "Error: Argument for $1 is missing" >&2
                exit 1
            fi
            ;;
        --tables)
            tables=$2
            shift 2
            ;;
        --all)
            all_flag=true
            shift
            ;;
        -*|--*=)
            echo "Error: Unsupported flag $1" >&2
            exit 1
            ;;
        *)
            shift
            ;;
    esac
done
```

```

# 加载数据到 Hive
if [[ "$all_flag" = true ]]; then
    for table in "${table_list[@]"; do
        hdfs_path="/origin_data/tms/${table:4}/$date"
        check_hdfs_file $hdfs_path
        load_data_to_hive $table $hdfs_path $date
    done
else
    if [[ -z "$tables" ]]; then
        echo "Error: You must specify either --tables or --all"
        exit 1
    fi
    IFS=' ' read -r -a array <<< "$tables"
    for table in "${array[@]"; do
        if [[ ! "${table_list[*]}" =~ "$table" ]]; then
            echo "Error: Unknown table $table"
            exit 1
        fi
        hdfs_path="/origin_data/tms/${table:4}/$date"
        check_hdfs_file $hdfs_path
        load_data_to_hive $table $hdfs_path $date
    done
fi

```

在运行这个脚本时，可以按照下面的方式提供命令行参数：

```
bash script.sh --date "2023-05-31" --tables "table1,table2"
```

或者：

```
bash script.sh --all
```

## 反馈

脚本已经写的非常好了，我们还可以优化一下，比如把里面涉及到的提示信息全部换成中文。

## 第 2 轮提示

### 提示

请将脚本中的提示信息全部换成中文，并加一个--help 的 flag，用以显示这个脚本的用法

### 答

好的，下面是更新后的脚本，已经将所有提示信息更改为中文，并增加了 --help 标志。

```
#!/bin/bash

# 获得昨天的日期
function get_yesterday_date() {
    date -d "yesterday" +%Y-%m-%d
}

# 检查 HDFS 路径下是否有文件
function check_hdfs_file() {
    hdfs dfs -test -e $1
    if [ $? -ne 0 ]; then
        echo "无此文件或目录: $1"
        exit 1
    fi
}

# 将数据 load 到 Hive 中
function load_data_to_hive() {
```

```

    tablename=$(echo $1 | cut -c5-)
    hive -e "LOAD DATA INPATH '$2' INTO TABLE $tablename PARTITION (dt='$3')"
}

# 声明配置变量 table_list
table_list=("table1" "table2" "table3")

# 默认的 date 为昨天
date=$(get_yesterday_date)

# 解析命令行参数
while (( "$#" )); do
    case "$1" in
        --help)
            echo "用法:"
            echo "--date: 可选参数, 指定日期, 默认为昨天。格式:yyyy-MM-dd"
            echo "--tables: 必选参数, 表名的列表, 接受逗号分隔的字符串, 例如:"
table1,table2,table3"
            echo "--all: 必选参数, 如果指定, 将加载所有表中的数据"
            exit 0
            ;;
        --date)
            if [ -n "$2" ] && [ ${2:0:1} != "-" ]; then
                date=$2
                shift 2
            else
                echo "错误: 参数 $1 缺少值" >&2
                exit 1
            fi
            ;;
        --tables)
            tables=$2
            shift 2
            ;;
        --all)
            all_flag=true

```

```

        shift
        ;;
    -*|--*=)
        echo "错误: 不支持的参数 $1" >&2
        exit 1
        ;;
    *)
        shift
        ;;
esac
done

# 加载数据到 Hive
if [[ "$all_flag" = true ]]; then
    for table in "${table_list[@]"; do
        hdfs_path="/origin_data/tms/${table:4}/$date"
        check_hdfs_file $hdfs_path
        load_data_to_hive $table $hdfs_path $date
    done
else
    if [[ -z "$tables" ]]; then
        echo "错误: 必须指定 --tables 或 --all"
        exit 1
    fi
    IFS=', ' read -r -a array <<< "$tables"
    for table in "${array[@]"; do
        if [[ ! " ${table_list[*]} " =~ " ${table} " ]]; then
            echo "错误: 未知的表 $table"
            exit 1
        fi
        hdfs_path="/origin_data/tms/${table:4}/$date"
        check_hdfs_file $hdfs_path
        load_data_to_hive $table $hdfs_path $date
    done
fi

```

你可以使用以下命令查看脚本用法：

```
bash script.sh --help
```

## 反馈

越来越好了。

## 习题三 辅助编写 SQL

### 第 1 轮提示

#### 提示

我正在使用 Hive 版本 3.1.2 进行数据分析，并需要处理以下任务：

源数据表（dws\_trans\_shift\_trans\_finish\_nd）是一个按班次粒度进行汇总的表。该表以 dt 字段作为分区，表示统计日期。字段 recent\_days 表示相对于统计日期的 1 天、7 天、或 30 天（这个字段只有这 3 种取值）。

我想从这个源数据表中计算以下指标：

最近 1/7/30 日各类卡车完成运输次数

最近 1/7/30 日各类卡车完成运输里程

最近 1/7/30 日各类卡车完成运输时长

最近 1/7/30 日各类卡车平均每次运输里程

最近 1/7/30 日各类卡车平均每次运输时长

并将计算结果写入另一张表（ads\_truck\_stats）中。

我需要一个 SQL 查询，用于抽取源数据表中的指定日期数据（例如，'2023-01-10'），并计算上述指标，然后将结果添加到目标表中。请注意，计算平均值时要避免分母为 0 的情况。

我的目标是得到一个可以针对任何给定日期执行的 SQL 查询，只需要简单地修改查询中的日期值。我希望你能帮我写出这个 SQL 查询。

下面是 dws\_trans\_shift\_trans\_finish\_nd 的建表语句

```
create external table dws_trans_shift_trans_finish_nd(  
  
    `shift_id` bigint comment '班次 ID',  
  
    `city_id` bigint comment '城市 ID',  
  
    `city_name` string comment '城市名称',  
  
    `org_id` bigint comment '机构 ID',  
  
    `org_name` string comment '机构名称',  
  
    `line_id` bigint comment '线路 ID',  
  
    `line_name` string comment '线路名称',  
  
    `driver1_emp_id` bigint comment '第一司机员工 ID',  
  
    `driver1_name` string comment '第一司机姓名',  
  
    `driver2_emp_id` bigint comment '第二司机员工 ID',  
  
    `driver2_name` string comment '第二司机姓名',  
  
    `truck_model_type` string comment '卡车类别编码',  
  
    `truck_model_type_name` string comment '卡车类别名称',  
  
    `recent_days` tinyint comment '最近天数',  
  
    `trans_finish_count` bigint comment '转运完成次数',  
  
    `trans_finish_distance` decimal(16,2) comment '转运完成里程',
```

```

`trans_finish_dur_sec` bigint comment '转运完成时长, 单位: 秒',

`trans_finish_order_count` bigint comment '转运完成运单数',

`trans_finish_delay_count` bigint comment '逾期次数'

) comment '物流域班次粒度转运完成最近 n 日汇总表'

partitioned by (`dt` string comment '统计日期')

```

下面是目标表的建表语句

```

create external table ads_truck_stats(

`dt` string COMMENT '统计日期',

`recent_days` tinyint COMMENT '最近天数, 1:最近天数, 1:最近 1 天, 7:最近 7 天, 30:最近 30 天',

`truck_model_type` string COMMENT '卡车类别编码',

`truck_model_type_name` string COMMENT '卡车类别名称',

`trans_finish_count` bigint COMMENT '完成运输次数',

`trans_finish_distance` decimal(16,2) COMMENT '完成运输里程',

`trans_finish_dur_sec` bigint COMMENT '完成运输时长, 单位: 秒',

`avg_trans_finish_distance` decimal(16,2) COMMENT '平均每次运输里程',

`avg_trans_finish_dur_sec` bigint COMMENT '平均每次运输时长, 单位: 秒'

COMMENT '各章节观看人数'

)

```

## 答

您可以使用以下 SQL 查询来完成此任务。这个查询是以给定日期（例如，'2023-01-10'）为参数，从源数据表（dws\_trans\_shift\_trans\_finish\_nd）中抽取数据，然后计算所需的指标，并将结果插入到目标表（ads\_truck\_stats）中。

请注意，当计算平均值时，我在 SQL 查询中加入了条件来避免除数为 0 的情况。如果转运完成次数（trans\_finish\_count）为 0，平均每次运输里程和运输时长将设为 NULL。

```
INSERT OVERWRITE TABLE ads_truck_stats PARTITION(dt='2023-01-10')
SELECT
    recent_days,
    truck_model_type,
    truck_model_type_name,
    SUM(trans_finish_count) AS trans_finish_count,
    SUM(trans_finish_distance) AS trans_finish_distance,
    SUM(trans_finish_dur_sec) AS trans_finish_dur_sec,
    CASE
        WHEN SUM(trans_finish_count) > 0 THEN SUM(trans_finish_distance) /
SUM(trans_finish_count)
        ELSE NULL
    END AS avg_trans_finish_distance,
    CASE
        WHEN SUM(trans_finish_count) > 0 THEN SUM(trans_finish_dur_sec) /
SUM(trans_finish_count)
        ELSE NULL
    END AS avg_trans_finish_dur_sec
FROM
    dws_trans_shift_trans_finish_nd
WHERE
    dt='2023-01-10'
GROUP BY
    recent_days,
    truck_model_type,
```

```
truck_model_type_name  
;
```

为了使查询能够适应任何给定日期，您只需要将查询中的日期值（这里是'2023-01-10'）替换为所需日期即可。以上 SQL 语句假设你的 Hive 设置允许分区重写。如果不允许，你可能需要将分区字段从 INSERT 语句中移除，并在 SELECT 语句中添加。

## 反馈

一步到位。我们只需要修改一下 `overwrite` 之前的 `select` 逻辑，让它 `union` 一下之前的数据就可以了。